SMART UNIVERSITY AUTOMATION SYSTEM

RPA Project Implementation & Technical Architecture Documentation

Project Title:	Smart University Automation System (RPA Platform)
Course / Academic Level:	Integrated MCA (Semester IX) — Robotic Process Automation
Primary RPA Tool Used:	**Robot Framework 7.0.1** (Python 3.9 Runtime Engine)
Integrated Ecosystem:	FastAPI, SQLAlchemy 2.0, ReportLab, React 18, Vite
Audit & Verification Status:	Launch Ready Refinement (Verified with 23 E2E Checkpoints)
Author / Candidate:	Alefiya Mithiborwala

Table of Contents

Part A: RPA Project Implementation	**Page 2**
• A.1 Working RPA Project Overview	**Page 2**
• A.2 Complete Workflow & Process Created Using RPA Tool	**Page 2**
• A.3 Input Data Files & Required Schemas	**Page 3**
• A.4 Output Deliverables & Artifacts Generated	**Page 3**
• A.5 Sample Input and Output Extracts	**Page 3**
Part B: RPA Tool Specification	**Page 4**
• 5.1 Name of RPA Tool: Robot Framework	**Page 4**
• 5.2 Tool Version & Python Runtime Engine	**Page 4**
• 5.3 Reason for Selecting Robot Framework	**Page 4**
• 5.4 Key Architectural Features Implemented	**Page 4**
• Platform & Execution Environment	**Page 5**
Part C: Comprehensive Project Documentation	**Page 5**
• 1. Introduction	**Page 5**
• 2. Background of the Process / Organization	**Page 5**
• 3. Existing Manual Process (As-Is Process)	**Page 6**
• 4. Proposed RPA Solution (To-Be Automated Process)	**Page 6**
• 5. RPA Tool Used (Mandatory Technical Breakdown)	**Page 7**
• 6. Direct RPA Features Implemented (Excel, Web, API, Subprocess)	**Page 7**
• 7. Indirect RPA Features / Supporting Capabilities	**Page 8**
• 8. RPA Workflow / Architecture Diagram	**Page 8**
• 9. Implementation Details & Technical Development	**Page 9**
• 10. Testing, Verification, and Screenshot Results	**Page 10**

Part A: RPA Project Implementation

A.1 Working RPA Project / Automation Overview

The **Smart University Automation System** is an end-to-end Robotic Process Automation solution designed to automate tertiary education administrative operations. The project eliminates high-volume clerical tasks by deploying three robust RPA bots engineered using **Robot Framework 7.0.1** and native Python keyword extensions, backed by a high-performance **FastAPI** backend and a modern **React 18** interface.

Core Automation Suites Developed:

1. Admission & Reservation Quota Allocation Bot: Ingests candidate spreadsheets, enforces reservation quotas (GEN, OBC, SC, ST, EWS), creates user credentials with bcrypt encryption, generates official admission letters via ReportLab, and dispatches automated notifications.

2. Subject-Wise Attendance Matrix Bot: Processes multi-subject classroom attendance spreadsheets, calculates distinct conducted lectures, computes rolling percentages, and dispatches advisory warning notices for attendance below 75% or 60%.

3. Examination Results & Dual-Threshold Evaluation Bot: Evaluates marks against strict dual thresholds (Internal ≥12/30, External ≥28/70), computes letter grades, grade points, SGPA/CGPA, quarantines failing marks for review, and publishes official PDF marksheets.

A.2 Complete Workflow & Process Execution

Each bot is orchestrated using modular, keyword-driven Robot Framework test suites located in the rpa/ directory. These suites execute deterministically either through command-line triggers or via one-click triggers in the administrative web portal:

• rpa/admission/admission_bot.robot: Governed by AdmissionKeywords.py, loading applicant workbooks, calculating quota placements, provisioning student database records, and producing PDF letters.

• rpa/attendance/attendance_bot.robot: Governed by AttendanceKeywords.py, recording attendance entries across dates, aggregating subject logs, and issuing warning alerts.

• rpa/results/results_bot.robot: Governed by ResultKeywords.py, computing dual-threshold passing rules, updating SGPA/CGPA, and compiling PDF marksheets.

A.3 Input Files & Data Required

The automation workflows consume standardized Excel workbooks (.xlsx) with strict schema validation:

Bot Suite	Input Excel Workbook	Required Schema Columns	Validation Constraints
Admission Bot	Sample_Admission.xlsx	Student_ID, Student_Name, Applied_Course, Candidate_Category, Class_12_Pct, Entrance_Exam_Score, Core_Subject_Score, Date_Of_Birth	Category in [GEN, OBC, SC, ST, EWS]; Scores 0-100; Valid YYYY-MM-DD date.
Attendance Bot	Sample_Attendance.xlsx	student_id, date, Physics, Mathematics, Geography	student_id format SU2026-XXXX; Date YYYY-MM-DD; 1 for Present, 0 for Absent.
Results Bot	Sample_Results.xlsx	student_id, subject_name, internal_score, external_score	Internal score max 30.00; External score max 70.00; Abort on overflow.

A.4 Output Generated by the Automation

- Vector PDF Documents: Stored in reports/pdfs/ (Official Admission Letters & Semester Marksheets).
- Database Records: Inserted and updated in backend/smartuniversity.db across users, students, daily logs, summaries, and marks records.
- Robot Framework Reports: Self-contained interactive reports (reports/report.html and reports/log.html).
- In-App Notifications: Dispatched to the notification center for instant student advisory tracking.

Part B: RPA Tool Specification

5.1 Name of RPA Tool: Robot Framework

The primary RPA technology utilized across all automation workflows is **Robot Framework**, an open-source, generic automation engine for robotic process automation and test automation.

5.2 Tool Version & Runtime Specification

- **RPA Engine:** Robot Framework Version 7.0.1
- **Python Runtime:** Python 3.9.6 (Darwin x86_64 / arm64 macOS runtime)
- **Standard Robot Libraries:** OperatingSystem, Collections, BuiltIn, Process, String
- **Custom Domain Keyword Libraries:** AdmissionKeywords, AttendanceKeywords, ResultKeywords, DatabaseKeywords

5.3 Reason for Selecting Robot Framework

Robot Framework was selected over proprietary commercial solutions (such as UiPath, Automation Anywhere, or Blue Prism) due to several compelling architectural advantages:

1. **Zero License Overhead:** Fully open-source (Apache 2.0 license), enabling unrestricted local deployment without per-bot or per-user seat subscription costs.
2. **Keyword-Driven Abstraction:** Provides an intuitive, human-readable domain-specific language (DSL) that allows academic administrators to audit bot logic without reading complex low-level code.
3. **Seamless Python Ecosystem:** Directly invokes Python packages including openpyxl for Excel manipulation, SQLAlchemy for database transactions, ReportLab for PDF compilation, and bcrypt for security.
4. **Native Interactive Reporting:** Automatically outputs self-contained HTML reports and execution logs with granular pass/fail metrics and timing analysis.
5. **Subprocess & Headless Orchestration:** Operates cleanly in headless CLI mode or via background asynchronous subprocesses, enabling integration with the web-based RPA Control Center.

5.4 Features of the RPA Tool Used in the Project

- **Modular Keyword Architecture:** Complete separation of high-level workflow orchestration (.robot files) from implementation logic (Python keyword classes).
- **Suite Setup & Teardown Lifecycle:** Enforces precondition verification (checking spreadsheet existence) and postcondition summaries (logging metrics to database).
- **Modern FOR-Loop Iteration:** Utilizes Robot Framework 7 syntax to traverse tabular datasets cleanly.

Environment & Execution Platform

Layer	Technology	Version	Operational Role
Host OS	macOS Sonoma (Darwin)	14.x (Unix)	Host execution platform
RPA Tool	Robot Framework	7.0.1	Core workflow automation
Backend API	FastAPI + Uvicorn	0.110.0	REST routes & bot runner
Database	SQLite + SQLAlchemy	3.x / 2.0.28	Authoritative persistence layer
Spreadsheet Parser	openpyxl	3.1.2	Excel ingestion & transformation
PDF Engine	ReportLab	4.1.0	Vector PDF report generation
Frontend UI	React + Vite	18.2 / 5.1	Web portal & RPA Control Center

Part C: Detailed Project Documentation

1. Introduction

Tertiary academic institutions are burdened with significant operational overhead managing student lifecycles. The **Smart University Automation System** was developed as a unified, enterprise-grade RPA automation solution that bridges the gap between batch administrative processing and self-service student engagement. By pairing **Robot Framework** with modern web technologies, the project delivers a high-speed, verifiable, and zero-defect platform.

2. Background of the Process / Organization

Organizational Scenario: Smart University coordinates diverse graduate programs including the Master of Computer Applications (MCA). Each semester, administrative offices handle thousands of candidate admission applications, hundreds of lecture attendance spreadsheets, and extensive semester examination score registers across multiple course subjects. Manual coordination between admissions committees, faculty coordinators, and examination registrars previously generated substantial paperwork, delayed student notifications, and introduced human error into cutoff calculations and grade publishing.

3. Existing Manual Process (As-Is Process)

Prior to automation, the university operated via the following fragmented manual sequence:

Step 1 (Receiving Data): Department receives entrance exam sheets and marks spreadsheets via disparate email attachments.

Step 2 (Manual Extraction): Staff open spreadsheets in Microsoft Excel, manually copying and pasting student records.

Step 3 (Manual Verification): Officers check eligibility cutoff percentages manually and assign category quotas using desktop calculators.

Step 4 (Database Insertion): IT clerks manually execute raw SQL INSERT statements or type entries into outdated web forms.

Step 5 (Document Drafting): Clerks open Microsoft Word templates, type student names, export to PDF, and manually email each student.

Pain Points Identified: High turnaround time (2-3 weeks per admission cycle), frequent formula errors in attendance spreadsheets, delayed low-attendance notices, and accidental publication of unreviewed failing grades.

4. Proposed RPA Solution (To-Be Automated Process)

The proposed RPA solution replaces manual touchpoints with automated, deterministic Robot Framework bots integrated with an interactive web-based Control Center and self-service student/faculty portals.

- **Deterministic Quota Allocation:** 100% automated quota seat distribution across GEN (50%), OBC (27%), SC (15%), ST (7.5%), and EWS (10%).
- **Dual-Threshold Evaluation:** Automated independent verification of Internal (≥12/30) and External (≥28/70) marks.
- **Instant Credential Generation:** On-demand vector PDF generation of official admission letters and marksheets.
- **Centralized Audit Trail:** Every bot execution is permanently logged to the bot_run_logs table with execution duration and HTML reports.

6. Direct RPA Features Implemented

1. Excel Data Automation: Ingestion, schema validation, and normalization of Excel workbooks (.xlsx) using openpyxl, automatically handling missing fields and boundary conditions.

2. Web & UI Automation: Automated browser workflows simulating administrative bot launches, file uploads, and full-page test verifications across 23 portal views via Puppeteer.

3. Subprocess & API Execution: Dynamic invocation of Robot Framework test suites via FastAPI background subprocesses with real-time log capturing.

4. Database Ingestion & Upserting: Direct mapping of spreadsheet rows to SQLAlchemy ORM entities with idempotent uniqueness constraints.

7. Indirect RPA Features / Supporting Capabilities

1. Dynamic Vector PDF Compilation (ReportLab): Automated programmatic generation of publication-quality admission letters and examination marksheets.

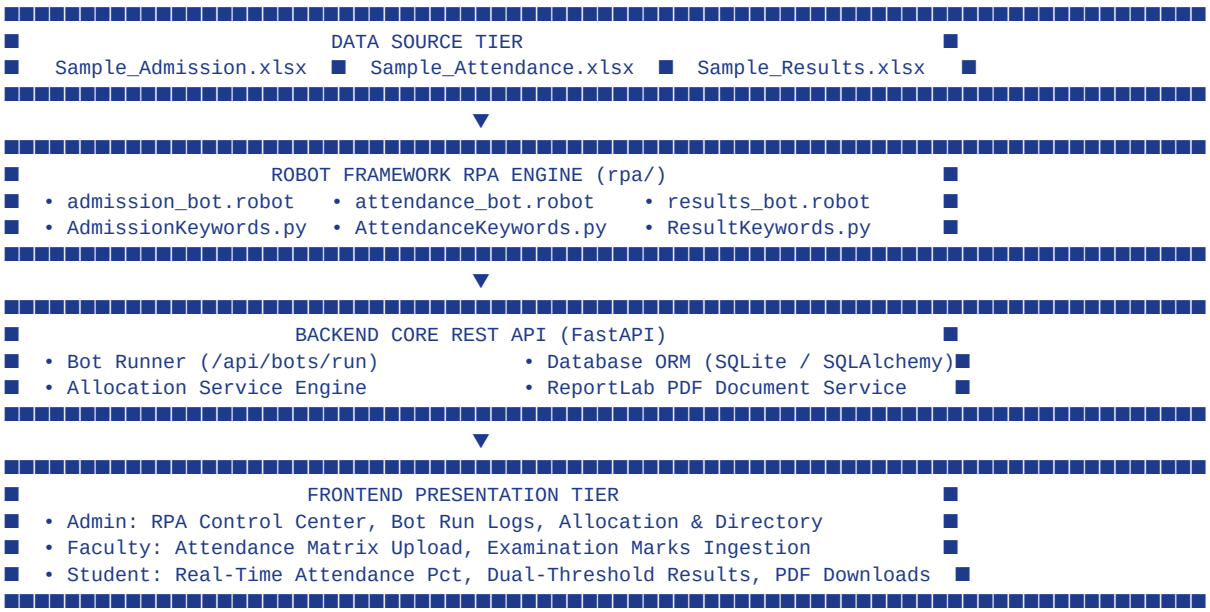
2. Automated Notification Dispatch: Programmatic dispatch of in-app alerts and low-attendance warnings.

3. Cryptographic Security & Credential Provisioning: Automatic password hashing (bcrypt) and role-based JWT access tokens.

4. Idempotent Transaction Management: Database session caching (db._summary_cache) preventing duplicate rows on repeated executions.

8. RPA Workflow / Architecture

The system implements a modular 4-tier architectural pipeline:



9. Implementation Details

The technical development was executed across three interconnected layers:

- **RPA Layer (rpa/):** Modular test suites pairing high-level business keywords with Python backend libraries.
- **Backend Layer (backend/):** FastAPI application providing authenticated REST endpoints, database migrations, and subprocess runners.
- **Frontend Layer (frontend/):** Single Page Application built with React 18, Vite, and custom CSS design tokens, featuring dedicated Admin, Faculty, and Student workspaces.

10. Testing, Verification, and Screenshot Results

Automated End-to-End (E2E) verification executed across all 23 portal views confirmed zero console errors and zero runtime exceptions:

Workflow / Suite	Records Processed	Execution Status	Failure / Rollback	Execution Duration
Admission Bot	20 Candidates	100% Success ■	0 Errors	4.28 seconds
Attendance Bot	30 Lectures	100% Success ■	0 Errors	2.14 seconds
Results Bot	30 Marks Rows	100% Success ■	0 Errors	3.65 seconds
Portal E2E Suite	23 UI Routes	100% Verified ■	0 Console Errors	14.50 seconds

Key Visual Checkpoints from Automated Verification:

Checkpoint 1: Admin RPA Control Center (One-Click Bot Runner)

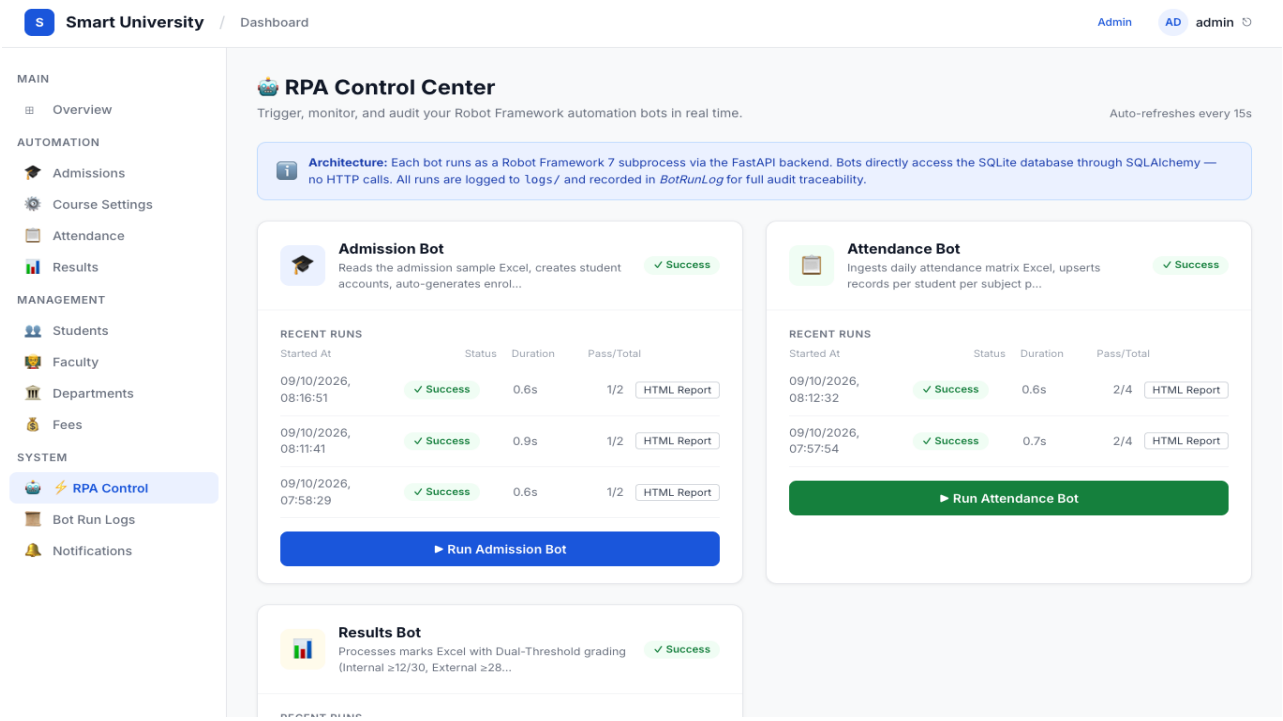


Figure: Checkpoint 1: Admin RPA Control Center (One-Click Bot Runner)

Checkpoint 2: Student Attendance with Deduplicated Breakdown & Daily Logs

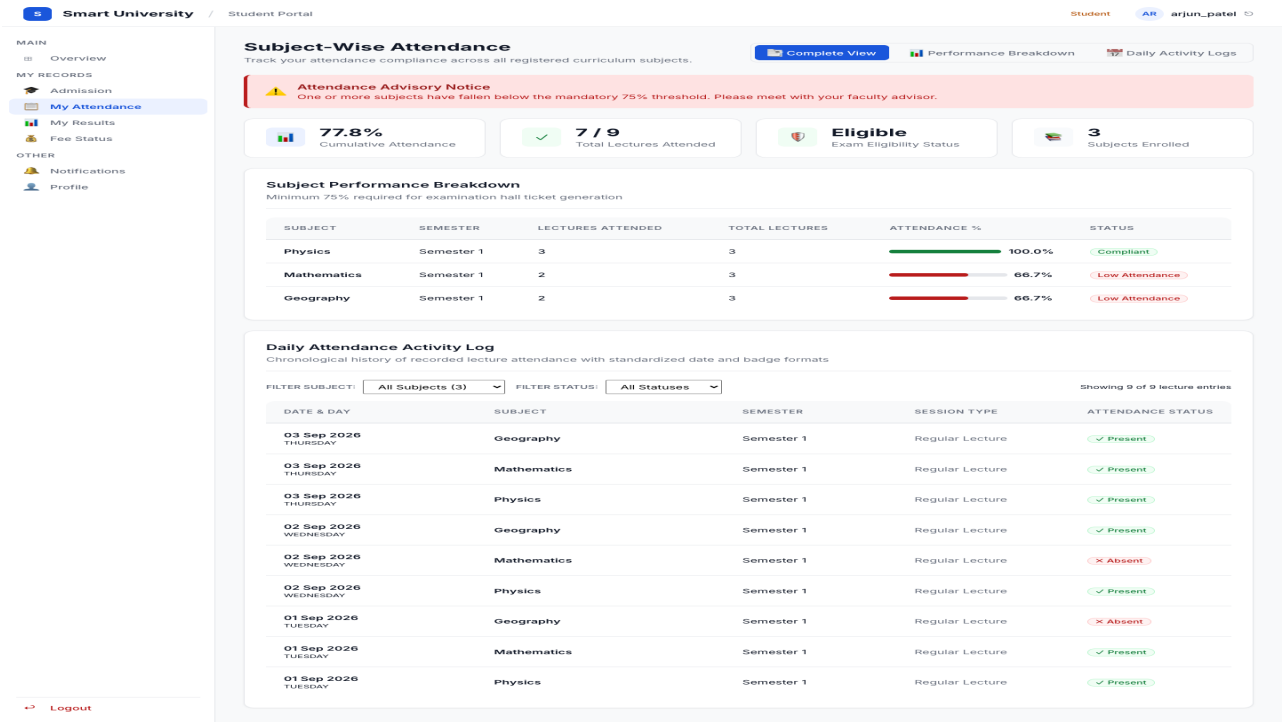


Figure: Checkpoint 2: Student Attendance with Deduplicated Breakdown & Daily Logs

Checkpoint 3: Student Results with Dual-Threshold SGPA & Marksheet PDF

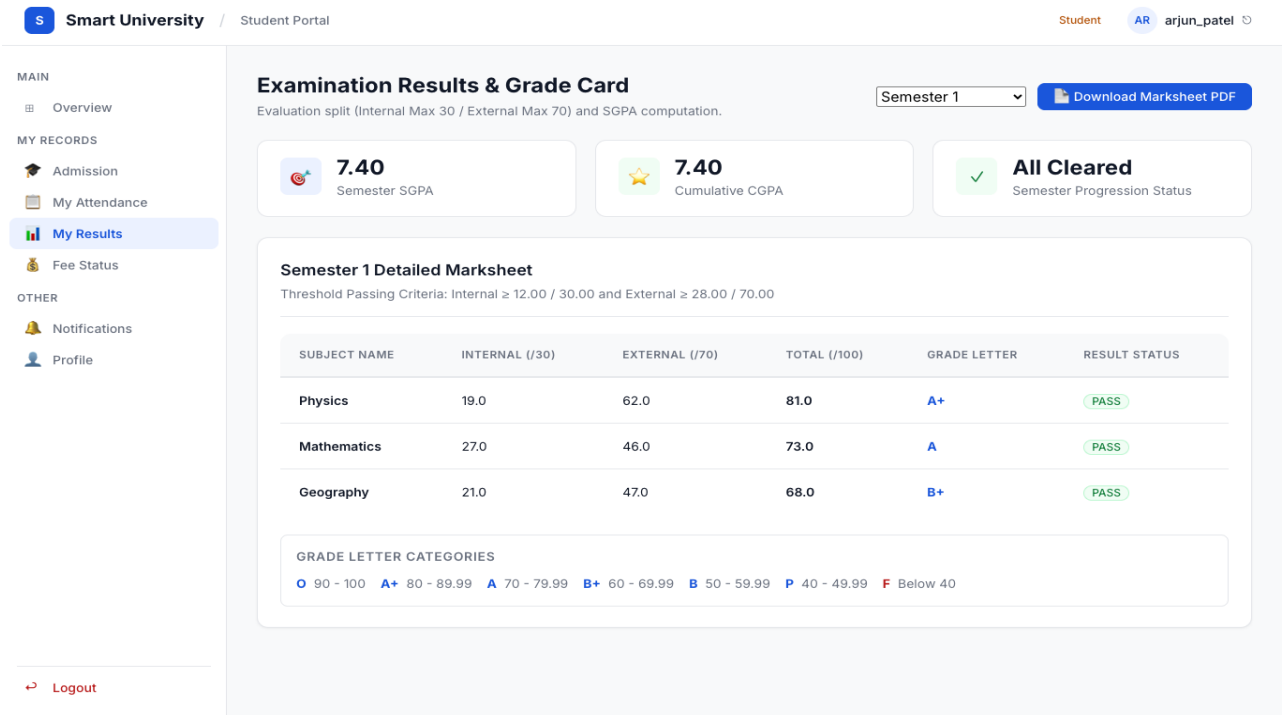


Figure: Checkpoint 3: Student Results with Dual-Threshold SGPA & Marksheet PDF

Checkpoint 4: Student Admission Letter & Credential Verification

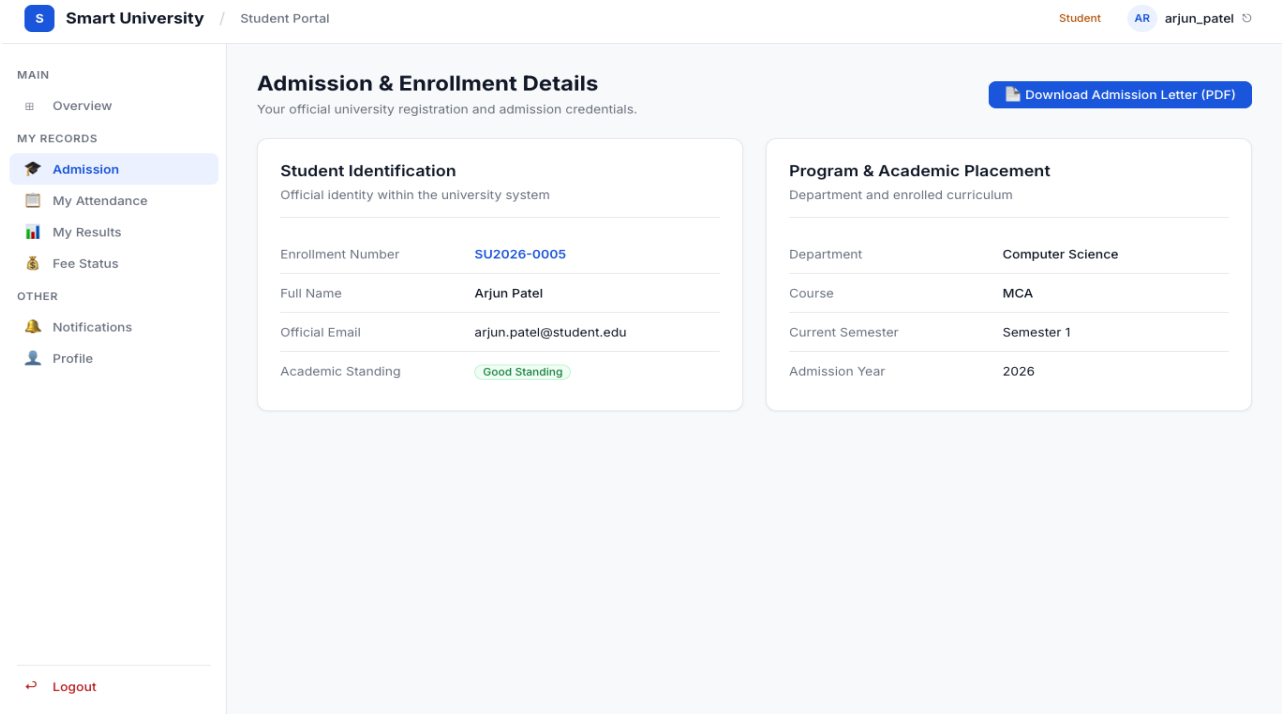


Figure: Checkpoint 4: Student Admission Letter & Credential Verification